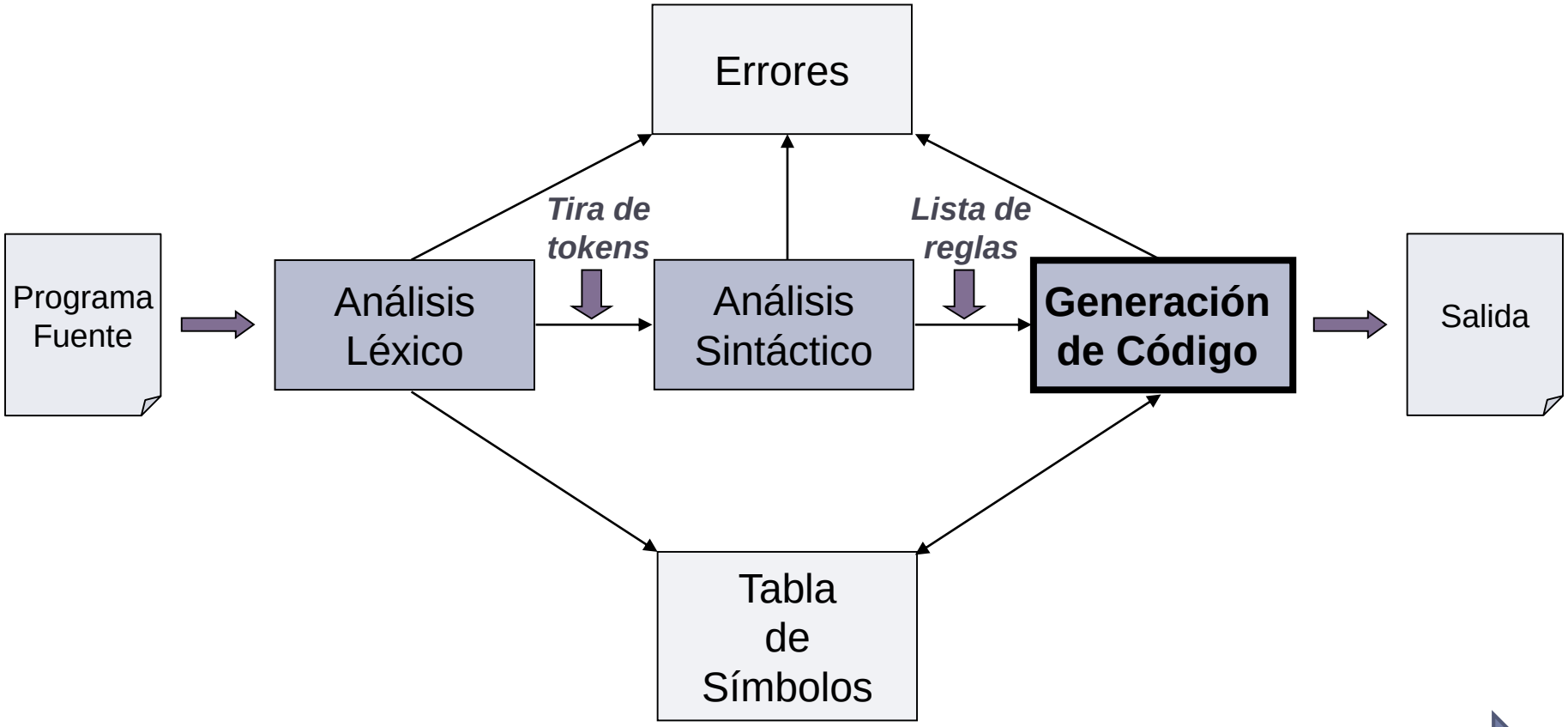
Diseño de Compiladores I
Compiladores e Intérpretes

Generación de Código

Fases de la Compilación



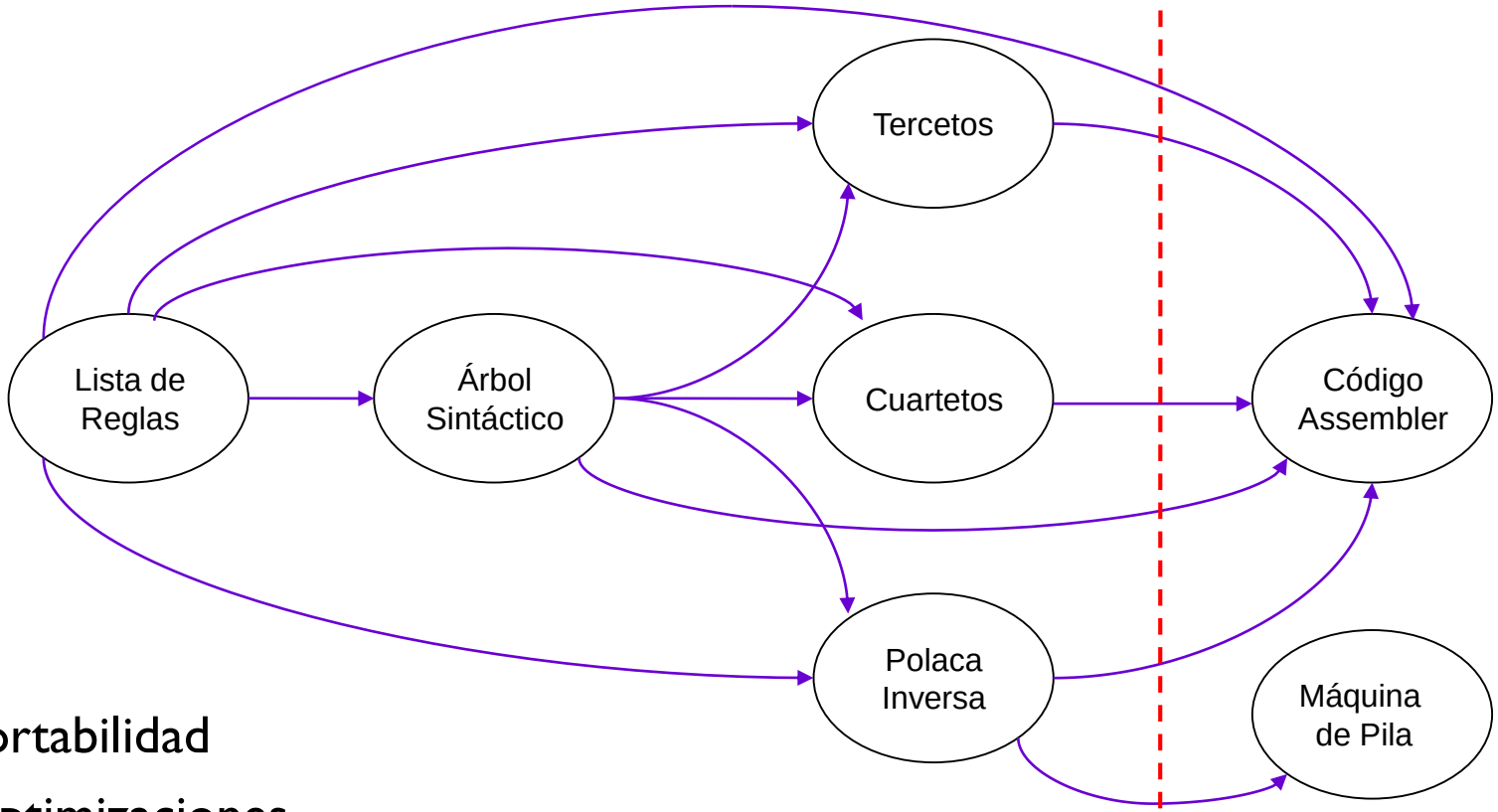
Saltear Repaso

Generación de Código
Grafo 1

Sentencias ejecutables
x := x + 2;

Sólo depende del lenguaje

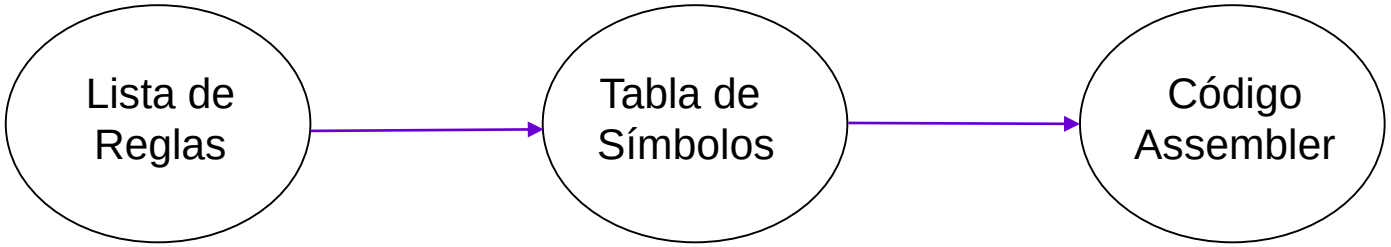
Depende del S.O. y de la máquina



- Portabilidad
- Optimizaciones

Generación de Código
Grafo 2: Sentencias declarativas

Sentencias declarativas
int x, y;



Expresiones – Asignaciones
(Repaso)

LISTA DE REGLAS → Árbol Sintáctico

LISTA DE REGLAS → Polaca Inversa

LISTA DE REGLAS → Tercetos

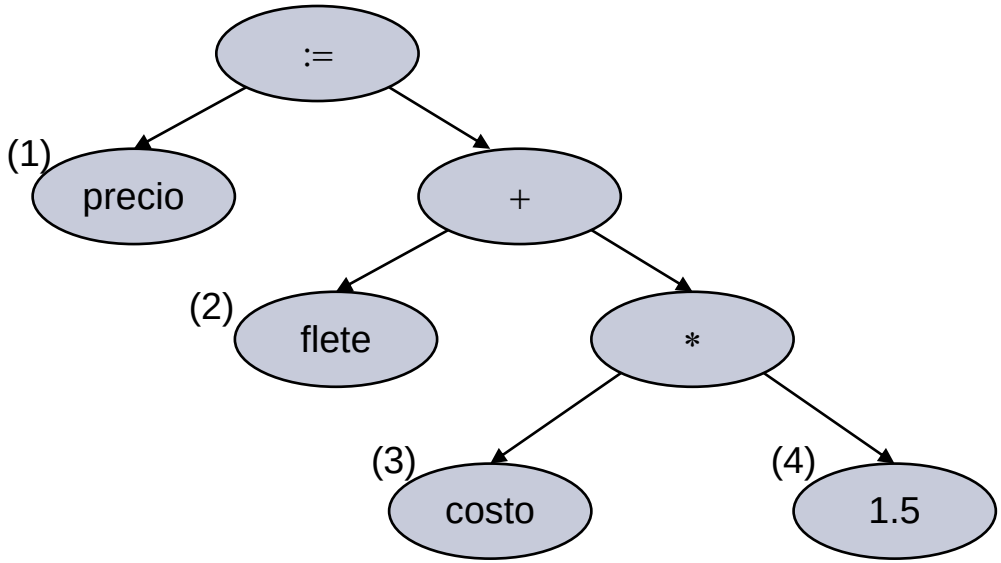
Representaciones:
Gramática

1) $A \rightarrow id := E$
2) $E \rightarrow E + T$
3) $E \rightarrow T$
4) $T \rightarrow T * F$
5) $T \rightarrow F$
6) $F \rightarrow id$
7) $F \rightarrow cte$



Árbol Sintáctico

precio := flete + costo * 1.5



Nota: Los nodos (1) (2) (3) y (4) contienen referencias a la Tabla de Símbolos

Lista de Reglas → Árbol Sintáctico

Reglas de la Gramática	Acciones Semánticas
1. $A \rightarrow id := E$	A.ptr = crear_nodo(':=' ; crear_hoja(id.ptr) ; E.ptr) (*)
2. $E \rightarrow E + T$	E.ptr = crear_nodo('+' ; E.ptr ; T.ptr)
3. $E \rightarrow T$	E.ptr = T.ptr;
4. $T \rightarrow T * F$	T.ptr = crear_nodo('*' ; T.ptr ; F.ptr)
5. $T \rightarrow F$	T.ptr = F.ptr;
6. $F \rightarrow id$	F.ptr = crear_hoja(id.ptr) (*)
7. $F \rightarrow cte$	F.ptr = crear_hoja(cte.ptr) (**)

(*) id.ptr = Ref a la entrada del identificador en la Tabla de Símbolos
(**) cte.ptr = Ref a la entrada de la constante en la Tabla de Símbolos

Tercetos

precio := flete + costo * 1.5

14. ...
15. (* , costo , 1.5)
16. (+ , flete , [15])
17. (:= , precio , [16])
18. ...

Los operandos *costo*, *flete*, *precio* y *1.5* son referencias a la Tabla de Símbolos

Los operandos [15] y [16] son referencias a los correspondientes tercetos

Lista de Reglas → Tercetos

Reglas de la Gramática	Acciones Semánticas
1. A → id := E	A.ptr = crear_terceto(':=' ; id.ptr ; E.ptr) (*)
2. E → E + T	E.ptr = crear_terceto('+' ; E.ptr ; T.ptr)
3. E → T	E.ptr = T.ptr;
4. T → T * F	T.ptr = crear_terceto('*' ; T.ptr ; F.ptr)
5. T → F	T.ptr = F.ptr;
6. F → id	F.ptr = id.ptr (*)
7. F → cte	F.ptr = cte.ptr (**)

(*) id.ptr = Ref a la entrada del identificador en la Tabla de Símbolos
(**) cte.ptr = Ref a la entrada de la constante en la Tabla de Símbolos

Polaca Inversa

precio := flete + costo * 1.5

...	flete	costo	1.5	*	+	precio	:=	...
21	22	23	24	25	26	27	28	29

Los operandos *costo*, *flete*, *precio* y *1.5* son referencias a la Tabla de Símbolos

Lista de Reglas → Polaca Inversa

Reglas de la Gramática	Acciones Semánticas
1. A → id := E	Agregar(id.ptr);Agregar(':=') (*)
2. E → E + T	Agregar('+')
3. E → T	-
4. T → T * F	Agregar('*')
5. T → F	-
6. F → id	Agregar(id.ptr) (*)
7. F → cte	Agregar(cte.ptr) (**)

(*) id.ptr = Ref a la entrada del identificador en la Tabla de Símbolos
(**) cte.ptr = Ref a la entrada de la constante en la Tabla de Símbolos

Sentencias de Control (Repaso)

LISTA DE REGLAS → Árbol Sintáctico

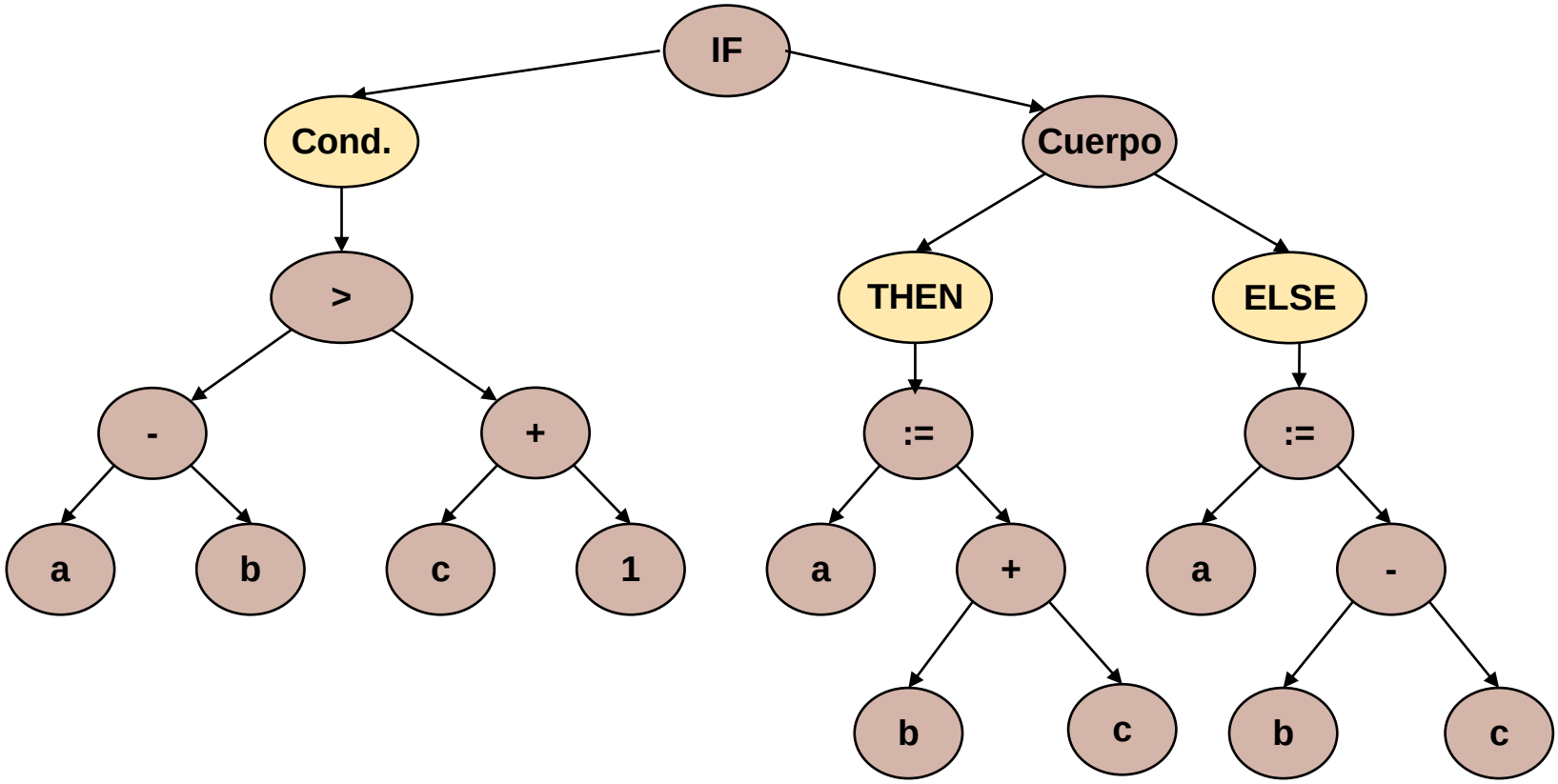
LISTA DE REGLAS → Polaca Inversa

LISTA DE REGLAS → Tercetos

Lista de Reglas → Árbol Sintáctico

Sentencia IF

IF (a – b > c + 1) THEN a := b + c; ELSE a := b – c;



El árbol tiene nodos de control para el algoritmo de generación de bifurcaciones

Lista de Reglas → Árbol Sintáctico

<seleccion> → IF <condicion> THEN <cpo_then> ELSE <cpo_else>



Reglas de la Gramática	Acciones Semánticas
<sel> → IF <cond> <cpo_if>	IF.ptr = crear_nodo("IF",C.ptr,C_I.ptr);
<cond> → '(' <exp_log> ')'	**// Agregar acciones para:** **// Crear el nodo de control "cond"**
<cpo_if> → <cpo_then> <cpo_else>	C_I.ptr = crear_nodo("cuerpo", C_T.ptr, C_E.ptr);
<cpo_if> → <cpo_then>	C_I.ptr = crear_nodo("cuerpo", C_T.ptr, null);
<cpo_then> → THEN <lista_sent>	**// Agregar acciones para:** **// Crear el nodo de control "then"**
<cpo_else> → ELSE <lista_sent>	**// Agregar acciones para:** **// Crear el nodo de control "else"**
<lista_sent> → …	L_S.ptr = …
<exp_log> → …	E_L.ptr = …

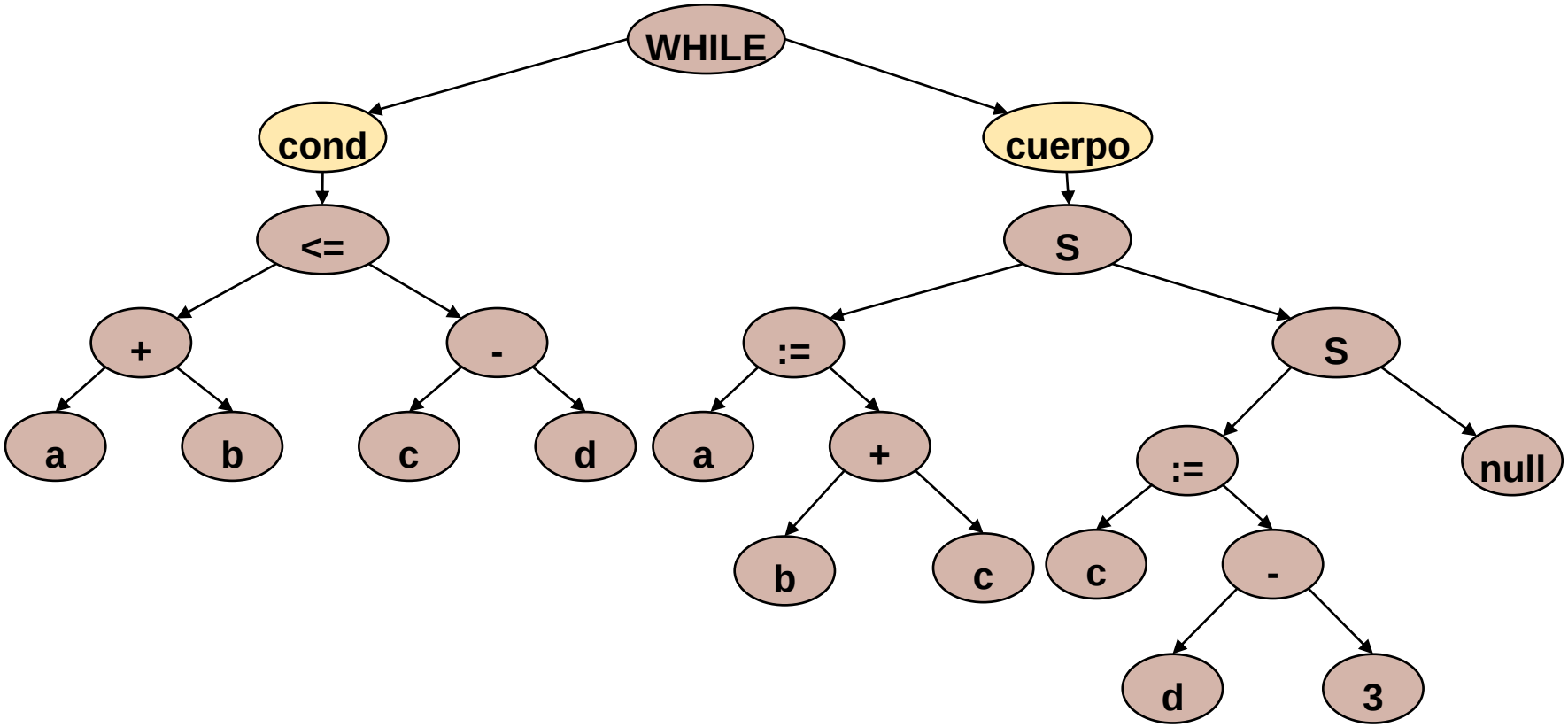


Lista de Reglas → Árbol Sintáctico

Sentencia WHILE

Ejemplo:

WHILE (a + b <= c - d) { a := b + c; c := d – 3;}



El árbol tiene nodos de control para el algoritmo de generación de bifurcaciones

Lista de Reglas → Árbol Sintáctico

<iteracion> → WHILE <condicion> <cpo_while>

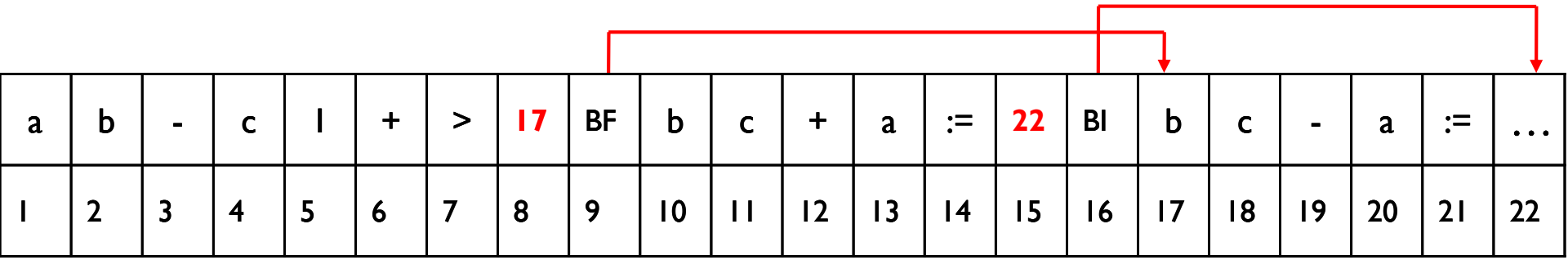
Reglas de la Gramática	Acciones Semánticas
<iter> → while <cond> <cpo_while>	I.ptr = crear_nodo("WHILE",C.ptr,C_W.ptr);
<cond> → '(' <exp_log> ')'	**// Agregar acciones para:** **// Crear el nodo de control "cond"**
<cpo_while> → <lista_sent>	**// Agregar acciones para:** **// Crear el nodo de control "cuerpo"**
<lista_sent> → …	L_S.ptr = …
<exp_log> → …	E_L.ptr = …

Lista de Reglas → Polaca Inversa

Sentencia IF

Ejemplo:

IF (a – b > c + 1) THEN a := b + c; ELSE a := b – c;



Backpatching

Lista de Reglas → Polaca Inversa

IF (a – b > c + 1) THEN a := b + c; ELSE a := b – c;

•Generar BF incompleta
•Apilar paso incompleto

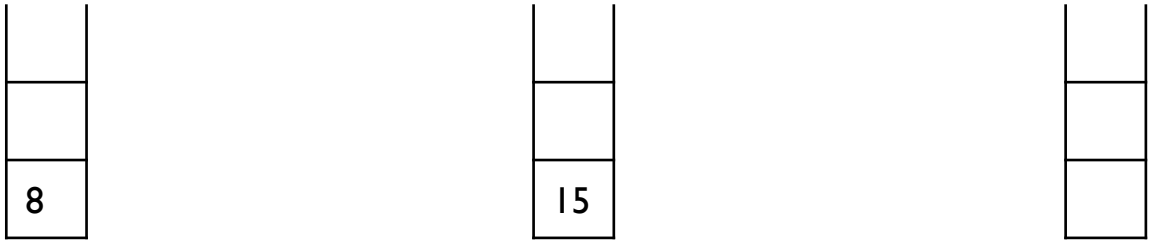
•Desapilar + Completar paso incompleto
•Generar BI con destino vacío
•Apilar paso incompleto

<seleccion> → IF <condicon> THEN <cpo_then> ELSE <cpo_else>

•Desapilar + Completar paso incompleto

Puntos de control para el algoritmo de generación de bifurcaciones

a	b	-	c	1	+	>	17	BF	b	c	+	a	:=	22	BI	b	c	-	a	:=	...
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22



Lista de Reglas → Polaca Inversa

<seleccion> → IF <condicon> THEN <cpo_then> ELSE <cpo_else>

Reglas de la Gramática	Acciones Semánticas

<sel> → IF <cond> <cpo_if>	// Agregar acciones para:
	// Desapilar
	// Completar el paso incompleto con el destino de la BI

<cond> → '(' <exp_log> ')'	// Agregar acciones para:
	// Generar los pasos de la BF incompleta
	// Apilar el número del paso incompleto

<cpo_if> → THEN <cpo_then> ELSE <cpo_else>	-
<cpo_if> → <cpo_then>	-

<cpo_then> → <lista_sent>	// Agregar acciones para:
	// Desapilar
	// Completar el paso incompleto con el destino de la BF
	// Generar los pasos de la BI incompleta
	// Apilar el número del paso incompleto

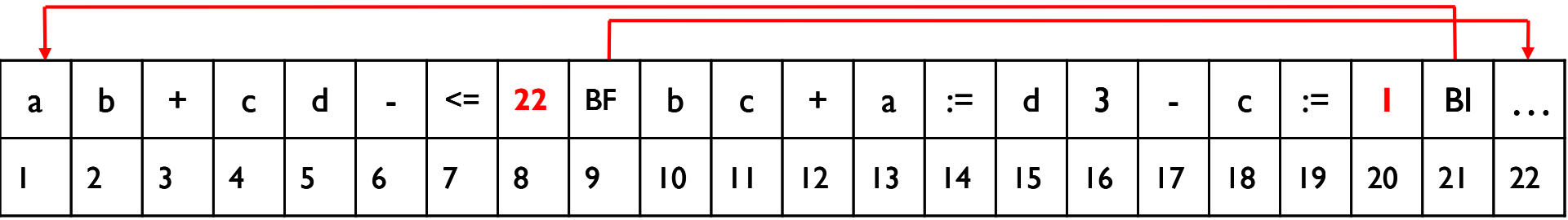
<cpo_else> → <lista_sent>	-
<lista_sent> → …	… *// Se crean los pasos para las sentencias de la lista*
<exp_log> → …	E_L.ptr = … *// Se crean los pasos de la expresión*

Lista de Reglas → Polaca Inversa

Sentencia WHILE

Ejemplo:

WHILE (a + b <= c - d) { a := b + c; c := d – 3;};

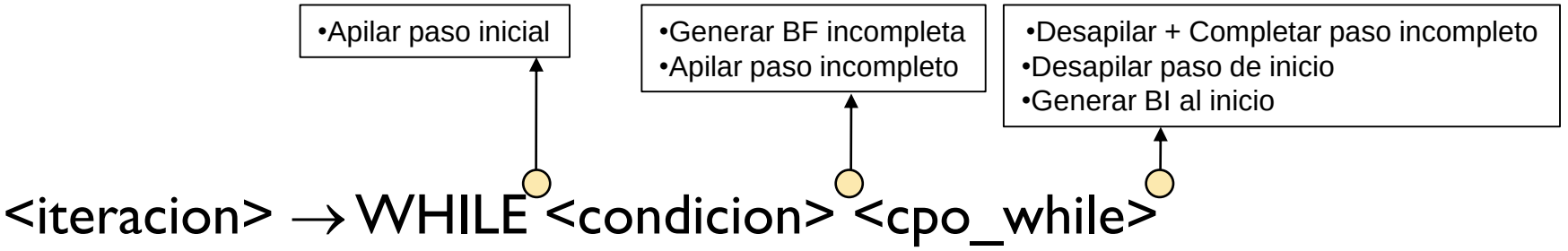


a	b	+	c	d	-	<=	22	BF	b	c	+	a	:=	d	3	-	c	:=	I	BI	...
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22

Backpatching

Lista de Reglas → Polaca Inversa

WHILE (a + b <= c - d) { a := b + c; c := d – 3; }



○ Puntos de control para el algoritmo de generación de bifurcaciones

a	b	+	c	d	-	<=	22	BF	b	c	+	a	:=	d	3	-	c	:=	1	BI	...
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22



Lista de Reglas → Polaca Inversa

<iteracion> → WHILE <condicion> <cpo_while>

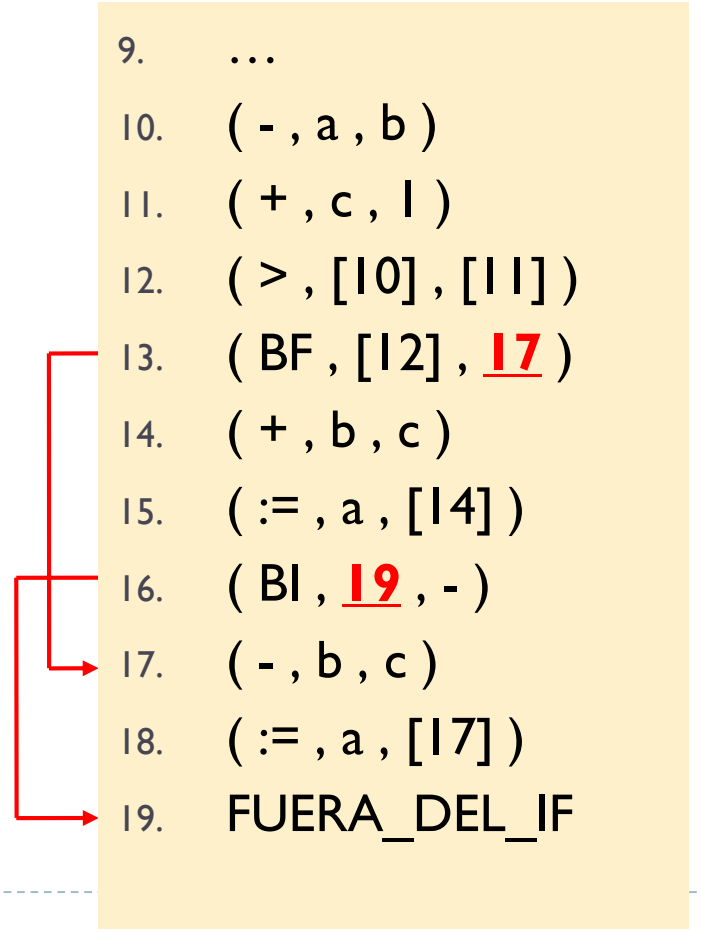
Reglas de la Gramática	Acciones Semánticas
<iter> → <inicio_while> <cond> <cpo_while>	**// Agregar acciones para:** **// Desapilar** **// Completar el paso incompleto con el destino de la BF** **// Desapilar paso de inicio** **// Generar los pasos de la BI**
<inicio_while> → WHILE	**// Agregar acciones para:** **// Apilar el número del paso donde comienza la condición**
<cond> → '(' <exp_log> ')'	**// Agregar acciones para:** **// Generar los pasos de la BF incompleta** **// Apilar el número del paso incompleto**
<cpo_while> → <lista_sent>	-
<lista_sent> → …	… ***// Se crean los pasos para las sentencias de la lista***
<exp_log> → …	E_L.ptr = … ***// Se crean los pasos de la expresión***

Lista de Reglas → Tercetos

Sentencia IF

Ejemplo:

IF (a – b > c + 1) THEN a := b + c; ELSE a := b – c;



Backpatching

Lista de Reglas → Tercetos

Ejemplo:

IF (a – b > c + 1) THEN a := b + c; ELSE a := b – c;

9. ...
10. (- , a , b)
11. (+ , c , 1)
12. (> , [10] , [11])
13. (BF , [12] , **17**)
14. (+ , b , c)
15. (:= , a , [14])
16. (BI , **19** , -)
17. (- , b , c)
18. (:= , a , [17])
19. FUERA_DEL_IF

- Generar BF incompleta
- Apilar terceto incompleto

- Desapilar + Completar terceto incompleto
- Generar BI con destino vacío
- Apilar terceto incompleto

- Desapilar + Completar terceto incompleto

Lista de Reglas → Tercetos

<seleccion> → IF <condicon> THEN <cpo_then> ELSE <cpo_else>



Reglas de la Gramática	Acciones Semánticas

<sel> → IF <cond> <cpo_if>	// Agregar acciones para:
	// Desapilar
	// Completar el terceto incompleto con el destino de la BI

<cond> → '(' <exp_log> ')'	// Agregar acciones para:
	// Crear terceto incompleto para la BF
	// Apilar el número del terceto incompleto

<cpo_if> → <cpo_then> ELSE <cpo_else>	-
<cpo_if> → <cpo_then>	-

<cpo_then> → <lista_sent>	// Agregar acciones para:
	// Desapilar
	// Completar el terceto incompleto con el destino de la BF
	// Crear terceto incompleto para la BI
	// Apilar el número del terceto incompleto

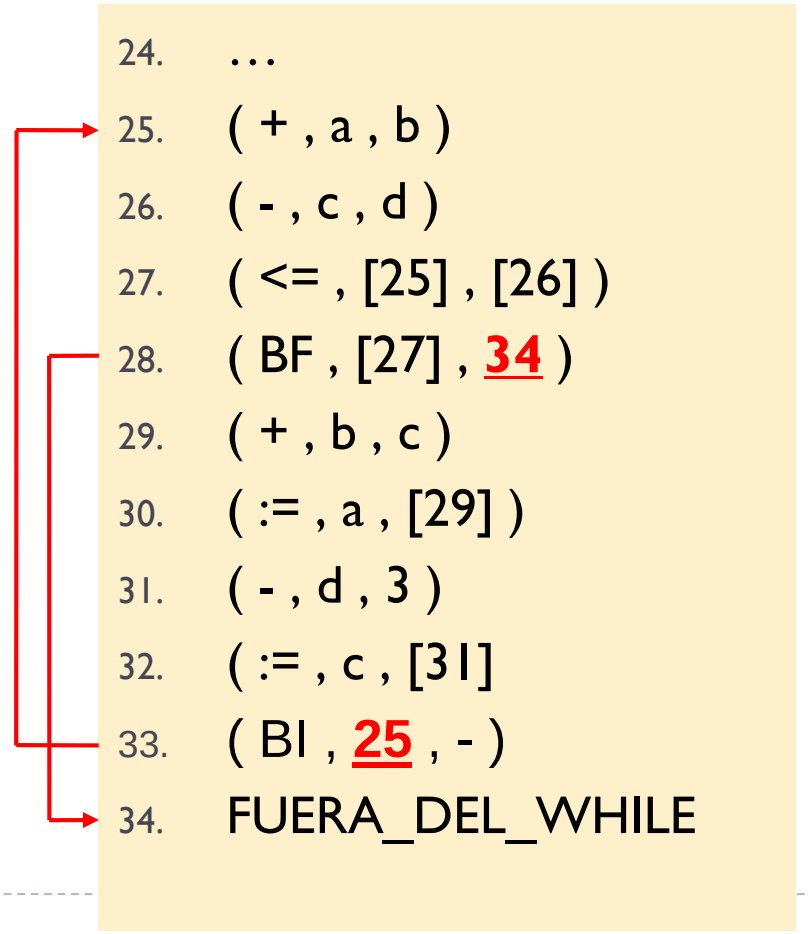
<cpo_else> → <lista_sent>	-
<lista_sent> → …	… // Se crean los tercetos para las sentencias de la lista
<exp_log> → …	E_L.ptr = … // Se crean los tercetos de la expresión

Lista de Reglas → Tercetos

Sentencia WHILE

Ejemplo:

WHILE (a + b <= c - d) { a := b + c; c := d – 3;};



Backpatching

Lista de Reglas → Tercetos

Ejemplo:

WHILE (a + b <= c - d) { a := b + c; c := d – 3;};

24. …
25. (+ , a , b)
26. (- , c , d)
27. (<= , [25] , [26])
28. (BF , [27] , **34**)
29. (+ , b , c)
30. (:= , a , [29])
31. (- , d , 3)
32. (:= , c , [31]
33. (BI , **25** , -)
34. FUERA_DEL_WHILE

- Apilar terceto inicial
- Generar BF incompleta
- Apilar terceto incompleto
- Desapilar + Completar terceto incompleto
- Desapilar dir. terceto de inicio
- Generar BI al inicio



Lista de Reglas → Tercetos

<iteracion> → WHILE <condicion> <cpo_while>

Reglas de la Gramática	Acciones Semánticas
<iter> → <inicio_while> <cond> <cpo_while>	**// Agregar acciones para:** **// Desapilar** **// Completar el terceto incompleto con el destino de la BF** **// Desapilar número de terceto de inicio** **// Crear el terceto para la BI**
<inicio_while> → WHILE	**// Agregar acciones para:** **// Apilar el número del terceto donde comienza la condición**
<cond> → '(' <exp_log> ')'	**// Agregar acciones para:** **// Crear terceto incompleto para la BF** **// Apilar el número del terceto incompleto**
<cpo_while> → <lista_sent>	-
<lista_sent> → …	… ***// Se crean los tercetos para las sentencias de la lista***
<exp_log> → …	E_L.ptr = … ***// Se crean los tercetos de la expresión***

Generación de Código

Traducción Dirigida por la Sintaxis

Traducción dirigida por la Sintaxis

- Se asocia información a los símbolos de la gramática (Terminales y No Terminales), mediante reglas semánticas asociadas a las producciones.
- Cada símbolo tendrá un conjunto de atributos asociados.
- Atributos:
  - cadena de caracteres (lexema),
  - tipo,
  - valor numérico (para optimizaciones, por ejemplo),
  - Información de ámbito,
  - etc.
- Los atributos de los símbolos calculados durante la Generación de Código, se utilizan para el cálculo de los atributos de otros símbolos.
- Para los símbolos Terminales, los atributos calculados se registran en la ***Tabla de Símbolos.***

Producciones + Reglas semánticas → Dependencias entre los atributos



Definiciones dirigidas por la Sintaxis

- El valor del atributo de un nodo de un Árbol de Parsing se define mediante una regla semántica asociada a la producción donde se genera ese nodo.

- Atributos Sintetizados
  - Se calculan a partir de los valores de los atributos de los hijos del nodo en el Árbol de Parsing.
- Atributos Heredados
  - Se calculan a partir de los valores de los atributos de los hermanos y el padre del nodo en el Árbol de Parsing.
  - Expresan dependencias entre una construcción y su contexto.



Atributos Sintetizados

- Se calculan a partir de los valores de los atributos de los hijos del nodo en el Árbol de Parsing.
- Ejemplo: Calculadora de escritorio

Reglas de la Gramática	Regla Semántica
1. L → E '\n'	Print (E.valor)
2. E → E + T	E.valor := E.valor + T.valor
3. E → T	E.valor = T.valor
4. T → T * F	T.valor := T.valor * F.valor
5. T → F	T.valor := F.valor
6. F → (E)	F.valor := E.valor
7. F → num	F.valor := num.valor

Atributo sintetizado: **valor**

El valor para el token num es proporcionado por el Analizador Léxico

- Los atributos sintetizados se calculan fácilmente en Parsing Ascendente.

Atributos Heredados

- Se calculan a partir de los valores de los atributos de los hermanos y el padre del nodo en el Árbol de Parsing.
- Ejemplo: Distribuir información de tipos a los identificadores de una declaración

Reglas de la Gramática	Regla Semántica
1. D → T L	L.tipo := T.tipo;
2. T→ int	T.tipo := integer:
3. T→ real	T.tipo := real;
4. L→ id ‘,’ L_1	añadir_tipo(id.ptr, L.tipo); L_1.tipo := L.tipo;
5. L → id	añadir_tipo(id.ptr, L.tipo);

Atributo heredado: **tipo**

Referencia a la entrada de id en la Tabla de Símbolos (Proporcionada por el A. Léxico)

- Los atributos heredados se calculan fácilmente en Parsing Descendente.



Chequeos Semánticos

Semántica Estática

- Se pueden efectuar en tiempo de compilación
- Ejemplos:
  - Comprobación de tipos
  - Comprobación de flujo de control
  - Comprobación de unicidad
  - Comprobaciones relacionadas con nombres

Semántica Dinámica

- Sólo se pueden efectuar en tiempo de ejecución
- Ejemplos:
  - Control de subíndices de arreglos



Generación de código

Lista de Reglas → Árbol Sintáctico
Lista de Reglas → Tercetos
Lista de Reglas → Polaca Inversa

Tipos y Conversiones

Tipos

$w := x + y * z$

Con:

w de tipo double

x de tipo integer

y de tipo float

z de tipo long

¿Es válida la expresión?

¿Es válida la asignación?



¿Cómo se detecta una incompatibilidad de tipos?



Tablas de Compatibilidad de Tipos

f_+	integer	long	float	double	string
integer	integer	long	float	double	X
long	long	long	float	double	X
float	float	float	float	double	X
double	double	double	double	double	X
string	X	X	X	X	string

- Si una combinación está prohibida, en la celda correspondiente habrá una "X", o un código de error para esa situación
- Si una combinación está permitida, en la celda correspondiente estará el tipo resultante,
- Para poder realizar una operación, ambos operandos deben ser compatibles en el momento de hacer la operación. Por lo tanto, es posible que uno o ambos operandos deban ser convertidos para poder hacer la operación.

Tablas de Tipos

- Habrá una matriz para cada operador de la gramática

f_+	integer	long	float	double	string
integer	integer	long	float	double	X
long	long	long	float	double	X
float	float	float	float	double	X
double	double	double	double	double	X
string	X	X	X	X	string

f_*	integer	long	float	double	string
integer	long	long	float	double	X
long	long	long	double	double	X
float	float	double	double	double	X
double	double	double	double	double	X
string	X	X	X	X	X

- Toda la semántica de los tipos está en la matriz

Tablas de Tipos

- Habrá una matriz para cada operador de la gramática

$F_{=}$	integer	long	float	double	string
integer	integer	X	X	X	X
long	long	long	X	X	X
float	float	float	float	X	X
double	double	double	double	double	X
string	X	X	X	X	string

- Toda la semántica de los tipos está en la matriz

Conversiones

Si una combinación está permitida, en la celda correspondiente estará el tipo resultante, y uno o ambos operandos deberán ser convertidos para poder hacer la operación.

- Implícitas (Coerciones)
  - Las realiza automáticamente el compilador.
  - En principio, se limitan a situaciones donde no hay pérdida de información.

- Explícitas
  - Las debe escribir el programador.



Tipos y Conversiones Implícitas

LISTA DE REGLAS → ÁRBOL SINTÁCTICO

Lista de Reglas → Árbol Sintáctico
(Información de tipos y conversiones implícitas)

w := x + y * z

w de tipo double

x de tipo integer

y de tipo float

z de tipo long

a) Árbol Sintáctico sin información de tipos ni incorporación de conversiones (implícitas)
b) Árbol Sintáctico con información de tipos
c) Árbol Sintáctico con información de tipos e incorporación de conversiones (implícitas)



Lista de Reglas → Árbol Sintáctico
(Información de tipos y conversiones implícitas)

w := x + y * z

w de tipo double

x de tipo integer

y de tipo float

z de tipo long

a) Árbol Sintáctico sin información de tipos ni incorporación de conversiones (implícitas)



Lista de Reglas → Árbol Sintáctico (a)
sin tipos ni conversiones

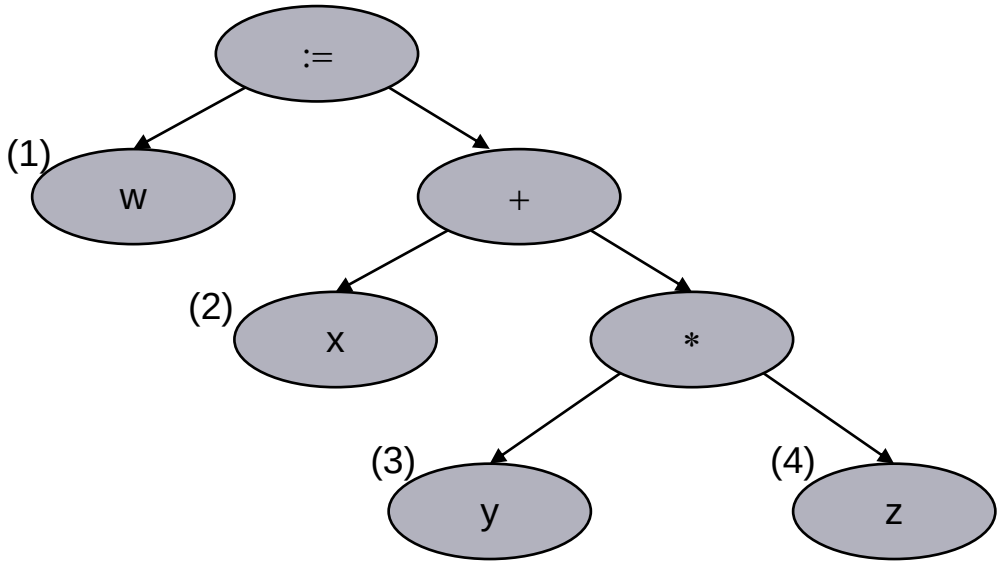
Reglas de la Gramática	Acciones Semánticas
1. A → id := E	A.ptr = crear_nodo(':=' ; crear_hoja(id.ptr) ; E.ptr) (*)
2. E → E + T	E.ptr = crear_nodo('+' ; E.ptr ; T.ptr)
3. E → T	E.ptr = T.ptr;
4. T → T * F	T.ptr = crear_nodo('*' ; T.ptr ; F.ptr)
5. T → F	T.ptr = F.ptr;
6. F → id	F.ptr = crear_hoja(id.ptr) (*)
7. F → cte	F.ptr = crear_hoja(cte.ptr) (**)

(*) id.ptr = Ref a la entrada del identificador en la Tabla de Símbolos
(**) cte.ptr = Ref a la entrada de la constante en la Tabla de Símbolos

Lista de Reglas → Árbol Sintáctico (a)
sin información de tipos ni incorporación de conversiones

w := x + y * z

w de tipo double
x de tipo integer
y de tipo float
z de tipo long



- El chequeo de tipos se posterga a una traducción posterior
- La incorporación de conversiones implícitas se posterga a una traducción posterior.



Lista de Reglas → Árbol Sintáctico

w := x + y * z

w de tipo double

x de tipo integer

y de tipo float

z de tipo long

b) Árbol Sintáctico con información de tipos



Lista de Reglas → Árbol Sintáctico (b)
con información de tipos

Reglas de la Gramática	Acciones Semánticas
1. $A \rightarrow id := E$	A.ptr = crear_nodo(':=' ; crear_hoja(id.ptr) ; E.ptr) ; (*) A.Tipo = id.tipo ;
2. $E \rightarrow E + T$	E.ptr = crear_nodo('+' ; E.ptr ; T.ptr) ; E.Tipo = f_+(E.tipo , T.tipo)
3. $E \rightarrow T$	E.ptr = T.ptr; E.tipo = T.tipo;
4. $T \rightarrow T * F$	T.ptr = crear_nodo('*' ; T.ptr ; F.ptr) ; T.tipo = f_*(T.tipo , F.tipo) ;
5. $T \rightarrow F$	T.ptr = F.ptr ; T.tipo = F.tipo ;
6. $F \rightarrow id$	F.ptr = crear_hoja(id.ptr) ; F.tipo = id.tipo; (*)
7. $F \rightarrow cte$	F.ptr = crear_hoja(cte.ptr) ; F.tipo = cte.tipo; (**)

Matriz de tipos

Matriz de tipos

(*) id.ptr = Ref a la entrada del identificador en la Tabla de Símbolos
(**) cte.ptr = Ref a la entrada de la constante en la Tabla de Símbolos

Tablas de Tipos

- Habrá una matriz para cada operador de la gramática

f_+	integer	long	float	double	string
integer	integer	long	float	double	X
long	long	long	float	double	X
float	float	float	float	double	X
double	double	double	double	double	X
string	X	X	X	X	string

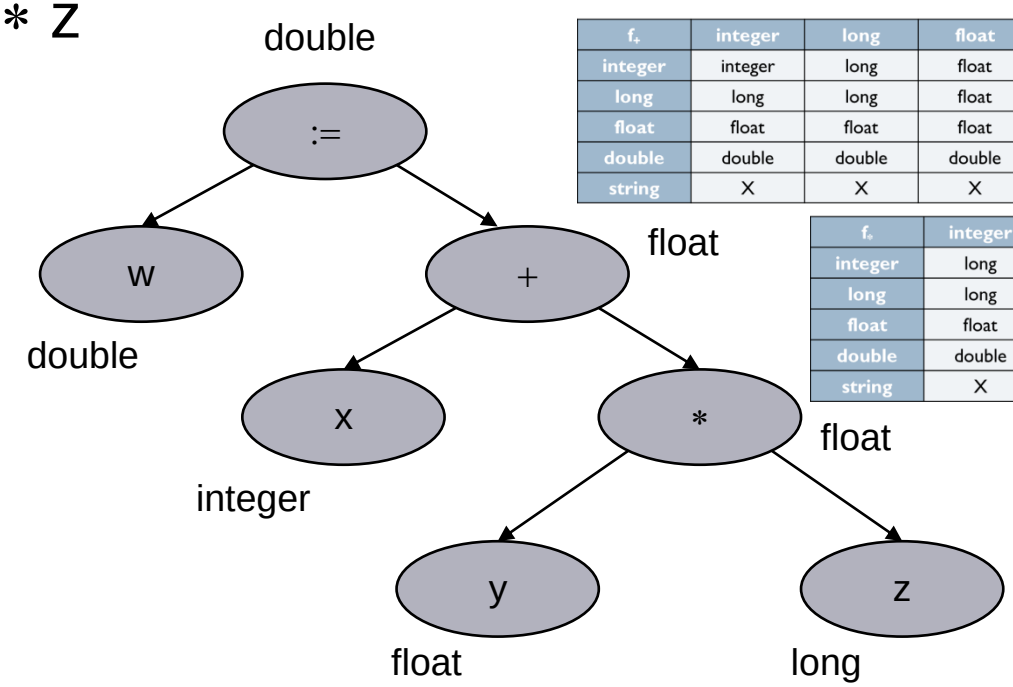
f_*	integer	long	float	double	string
integer	long	long	float	double	X
long	long	long	float	double	X
float	float	float	double	double	X
double	double	double	double	double	X
string	X	X	X	X	X

- Toda la semántica de los tipos está en la matriz

Lista de Reglas → Árbol Sintáctico (b)
con información de tipos

w := x + y * z

w de tipo double
x de tipo integer
y de tipo float
z de tipo long



f_+	integer	long	float	double	string
integer	integer	long	float	double	X
long	long	long	float	double	X
float	float	float	float	double	X
double	double	double	double	double	X
string	X	X	X	X	string

f_*	integer	long	float	double	string
integer	long	long	float	double	X
long	long	long	float	double	X
float	float	float	double	double	X
double	double	double	double	double	X
string	X	X	X	X	X

- La incorporación de conversiones implícitas se posterga a una traducción posterior.

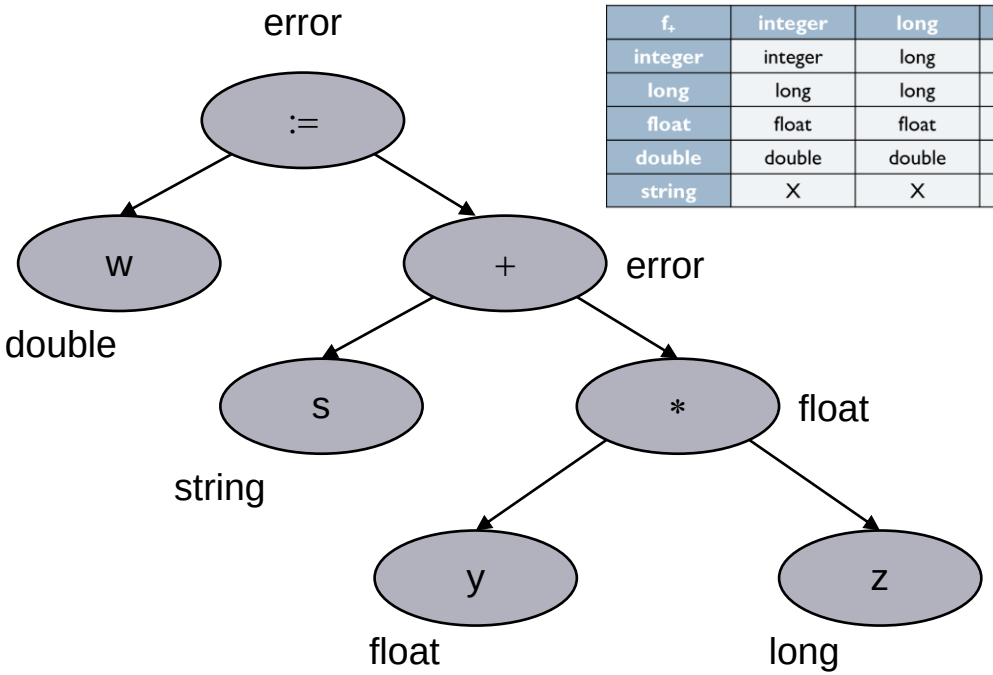


Lista de Reglas → Árbol Sintáctico (b)
con información de tipos

- Ejemplo con error:

w := s + y * z

w de tipo double
s de tipo string
y de tipo float
z de tipo long



f_+	integer	long	float	double	string
integer	integer	long	float	double	X
long	long	long	float	double	X
float	float	float	float	double	X
double	double	double	double	double	X
string	X	X	X	X	string

Lista de Reglas → Árbol Sintáctico

w := x + y * z

w de tipo double

x de tipo integer

y de tipo float

z de tipo long

c) Árbol Sintáctico con información de tipos e incorporación de conversiones (implícitas)



Tablas de Compatibilidad de Tipos

f_+	integer	long	float	double	string
integer	integer	long	float	double	X
long	long	long	float	double	X
float	float	float	float	double	X
double	double	double	double	double	X
string	X	X	X	X	string

- Si una combinación está prohibida, en la celda correspondiente habrá una "X", o un código de error para esa situación
- Si una combinación está permitida, en la celda correspondiente estará el tipo resultante,
- Para poder realizar una operación, ambos operandos deben ser compatibles en el momento de hacer la operación. Por lo tanto, es posible que uno o ambos operandos deban ser convertidos para poder hacer la operación.

Lista de Reglas → Árbol Sintáctico (b)

Tabla de Tipos y Conversiones

f_*	integer			long			float			double			string		
integer		**long**		itol	**long**	-	itof	**float**	-	itod	**double**	-	-	**X**	-
long	-	**long**	itol	-	**long**	-	ltof	**float**	-	ltod	**double**	-	-	**X**	-
float	-	**float**	itof	-	**float**	ltof	ftod	**double**	ftod	ftod	**double**	-	-	**X**	-
double	-	**double**	itod	.	**double**	ltod	-	**double**	ftod	-	**double**	-	-	**X**	-
string	-	**X**	-	-	**X**	-	-	**X**	-	-	**X**	-	-	**X**	-

- Toda la semántica de los tipos y ***conversiones*** está en la matriz
- Habrá una matriz para cada operador de la gramática
- Si una combinación está prohibida, en la celda correspondiente habrá una "X", o un código de error para esa situación

f_+	integer			long			float			double			string		
integer		**integer**		itol	**long**	-	itof	**float**	-	itod	**double**	-	-	**X**	-
long	-	**long**	itol	-	**long**	-	ltof	**float**	-	ltod	**double**	-	-	**X**	-
float	-	**float**	itof	-	**float**	ltof	-	**float**	-	ftod	**double**	-	-	**X**	-
double	-	**double**	itod	.	**double**	ltod	-	**double**	ftod	-	**double**	-	-	**X**	-
string	-	**X**	-	-	**X**	-	-	**X**	-	-	**X**	-	-	**string**	-

Lista de Reglas → Árbol Sintáctico (c)
con información de tipos e incorporación de conversiones

Reglas de la Gramática	Acciones Semánticas
1. $A \rightarrow id := E$	E'.ptr = agregar_nodo_conversion(f:=[3] ; id.tipo ; E.tipo); A.ptr = crear_nodo(':=' ; crear_hoja(id.ptr) ; E'.ptr) ; (*) A.Tipo = id.tipo ;
2. $E \rightarrow E + T$	E'.ptr = agregar_nodo_conversion(f+[1] ; E.tipo ; T.tipo); T'.ptr = agregar_nodo_conversion(f+[3]; E.tipo ; T.tipo); E.ptr = crear_nodo('+' ; E'.ptr ; T'.ptr) ; E.Tipo = f_+[2](E.tipo , T.tipo) ;
3. $E \rightarrow T$	E.ptr = T.ptr; E.tipo = T.tipo;
4. $T \rightarrow T * F$	T'.ptr = agregar_nodo_conversion(f*[1] ; T.tipo ; F.tipo); F'.ptr = agregar_nodo_conversion(f* [3]; T.tipo ; F.tipo); T.ptr = crear_nodo('*' ; T'.ptr ; F'.ptr) ; T.tipo = f_*[2](T.tipo , F.tipo) ;
5. $T \rightarrow F$	T.ptr = F.ptr ; T.tipo = F.tipo ;
6. $F \rightarrow id$	F.ptr = crear_hoja(id.ptr) ; F.tipo = id.tipo; (*)
7. $F \rightarrow cte$	F.ptr = crear_hoja(cte.ptr) ; F.tipo = cte.tipo; (**)

(*) id.ptr = Ref a la entrada del identificador en la Tabla de Símbolos (**) cte.ptr = Ref a la entrada de la constante en la Tabla de Símbolos

Lista de Reglas → Árbol Sintáctico (c)

con información de tipos e incorporación de conversiones

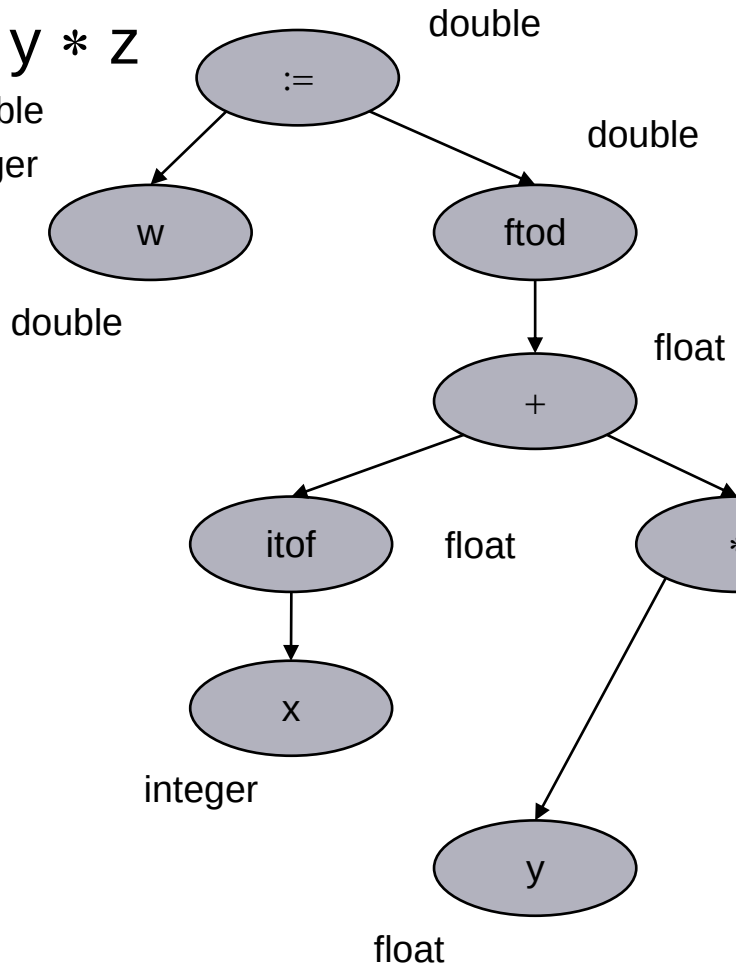
w := x + y * z

w de tipo double

x de tipo integer

y de tipo float

z de tipo long



f_+	integer	long	float	double	string
integer	integer	long	float	double	X
long	long	long	float	double	X
float	float	float	float	double	X
double	double	double	double	double	X
string	X	X	X	X	string

	integer	long	float	double	string
integer	integer	long	float	double	X
long	long	long	float	double	X
float	float	float	double	double	X
double	double	double	double	double	X
string	X	X	X	X	string



Tipos y Conversiones Implícitas

LISTA DE REGLAS → TERCETOS

Lista de Reglas → Tercetos
(Información de tipos y conversiones implícitas)

w := x + y * z

w de tipo double

x de tipo integer

y de tipo float

z de tipo long

a) Tercetos sin información de tipos ni incorporación de conversiones (implícitas)
b) Tercetos con información de tipos
c) Tercetos con información de tipos e incorporación de conversiones (implícitas)



a) Tercetos sin información de tipos ni incorporación de conversiones (implícitas)

w := x + y * z (w de tipo double, x de tipo integer, y de tipo float, z de tipo long)

```
20.  ...
21.  ( * , y , z )
22.  ( + , x , [21] )
23.  ( := , w , [22] )
24.  ...
```

Los operandos ***x***, ***y***, ***z*** *y* ***w*** son referencias a la Tabla de Símbolos

- El chequeo de tipos se posterga a una traducción posterior
- La incorporación de conversiones implícitas se posterga a una traducción posterior.



b) Tercetos con información de tipos)

w := x + y * z (w de tipo double, x de tipo integer, y de tipo float, z de tipo long)

f_*	integer	long	float	double	string
integer	long	long	float	double	X
long	long	long	float	double	X
float	float	float	double	double	X
double	double	double	double	double	X
string	X	X	X	X	X

f_+	integer	long	float	double	string
integer	integer	long	float	double	X
long	long	long	float	double	X
float	float	float	float	double	X
double	double	double	double	double	X
string	X	X	X	X	string

20. ...
21. (* , y , z) float
22. (+ , x , [21]) float
23. (:= , w , [22]) double
24. ...

Los operandos ***x***, ***y***, ***z*** *y* ***w*** son referencias a la Tabla de Símbolos

- La incorporación de conversiones implícitas se posterga a una traducción posterior.



c) Tercetos con información de tipos e incorporación de conversiones (implícitas)

w := x + y * z (w de tipo double, x de tipo integer, y de tipo float, z de tipo long)

20.	...	
21.	(ltof, z , -)	float
22.	(* , y , [21])	float
23.	(itof , x , -)	float
24.	(+ , [23] , [22])	float
25.	(ftod , [24] , -)	double
26.	(:= , w , [25])	
27.	...	

f_+	integer			long			float			double			string		
integer		long		itol	long	-	itof	float	-	itod	double	-	-	x	-
long	-	long	itol	-	long	-	ltof	float	-	ltod	double	-	-	x	-
float	-	float	itof	-	float	ltof	ftod	double	ftod	ftod	double	-	-	x	-
double	-	double	itod	.	double	ltod	-	double	ftod	-	double	-	-	x	-
string	-	x	-	-	x	-	-	x	-	-	x	-	-	x	-

f_+	integer			long			float			double			string		
integer		integer		itol	long	-	itof	float	-	itod	double	-	-	x	-
long	-	long	itol	-	long	-	ltof	float	-	ltod	double	-	-	x	-
float	-	float	itof	-	float	ltof	-	float	-	ftod	double	-	-	x	-
double	-	double	itod	.	double	ltod	-	double	ftod	-	double	-	-	x	-
string	-	x	-	-	x	-	-	x	-	-	x	-	-	string	-

Los operandos ***x***, ***y***, ***z*** y ***w*** son referencias a la Tabla de Símbolos

Tipos y Conversiones Implícitas

LISTA DE REGLAS → POLACA INVERSA

Lista de Reglas → Polaca Inversa
(Información de tipos y conversiones implícitas)

w := x + y * z

w de tipo double

x de tipo integer

y de tipo float

z de tipo long

(a) Polaca Inversa sin información de tipos ni incorporación de conversiones (implícitas)



Polaca Inversa

w := x + y * z (w de tipo double, x de tipo integer, y de tipo float, z de tipo long)

...	x	y	z	*	+	w	:=	...
21	22	23	24	25	26	27	28	29

Los operandos ***x***, ***y***, ***z*** *y* ***w*** son referencias a la Tabla de Símbolos

- El chequeo de tipos se **DEBE** postergar a una traducción posterior
- La incorporación de conversiones implícitas se **DEBE** postergar a una traducción posterior.



Tipos y Conversiones

ÁRBOL SINTÁCTICO → TERCETOS

ÁRBOL SINTÁCTICO → POLACA INVERSA

Árbol Sintáctico → Tercetos
(Información de tipos y conversiones implícitas)

(a) (a)

Árbol Sintáctico sin tipos ni conversiones → Tercetos sin tipos ni conversiones

(El chequeo de tipos y la incorporación de conversiones se posterga a la traducción Tercetos → Assembler)

(a) (b)

Árbol Sintáctico sin tipos ni conversiones → Tercetos con tipos

(La incorporación de conversiones se posterga a la traducción Tercetos → Assembler)

(a) (c)

Árbol Sintáctico sin tipos ni conversiones → Tercetos con tipos y conversiones

(b) (b)

Árbol Sintáctico con tipos → Tercetos con tipos

(La incorporación de conversiones se posterga a la traducción Tercetos → Assembler)

(b) (c)

Árbol Sintáctico con tipos → Tercetos con tipos y conversiones

(c) (c)

Árbol Sintáctico con tipos y conversiones → Tercetos con tipos y conversiones



Árbol Sintáctico → Polaca Inversa
(Información de tipos y conversiones implícitas)

(a) (a)

Árbol Sintáctico sin tipos ni conversiones → Polaca sin tipos ni conversiones

(El chequeo de tipos y la incorporación de conversiones se posterga a la traducción Polaca Inversa → Assembler)

(b) (b)

Árbol Sintáctico con tipos → Polaca con tipos

(La incorporación de conversiones se posterga a la traducción Polaca Inversa → Assembler)

(c) (c)

Árbol Sintáctico con tipos y conversiones → Polaca con tipos y conversiones



Tipos y Conversiones Explícitas

ÁRBOL SINTÁCTICO
TERCETOS
POLACA INVERSA

Conversiones explícitas

w := x + y * z

Con:

w de tipo long

x de tipo integer

y de tipo float

z de tipo long

¿Es válida la expresión?

¿Es válida la asignación?

Para un lenguaje que sólo admite conversiones explícitas, esta sentencia dará errores de incompatibilidad de tipos



Conversiones explícitas

w := x + **y * z**

¡Tipos incompatibles!

w de tipo long

x de tipo integer

y de tipo float

z de tipo long

- Para poder realizar una operación, ambos operandos deben ser compatibles al momento de hacer la operación.

f_*	integer	long	float	double	string
integer	long	X	X	X	X
long	X	long	X	X	X
float	X	X	double	X	X
double	X	X	X	double	X
string	X	X	X	X	X

- Si una combinación está permitida, en la celda correspondiente estará el tipo resultante.
- Para que la operación se pueda realizar, uno o los dos operandos deberán ser convertidos antes de efectuar la operación.



Conversiones explícitas

¡Tipos incompatibles!

$w := x + y * \text{tofloat}(z)$

x de tipo integer

El producto de y * tofloat(z)

es double

y de tipo float

z de tipo long

w de tipo long

F_+	integer	long	float	double	string
integer	integer	X	X	X	X
long	X	long	X	X	X
float	X	X	float	X	X
double	X	X	X	double	X
string	X	X	X	X	string



Conversiones explícitas



$$w := todouble(x) + y * tofloat(z)$$

w de tipo long

resultado de la expresión de tipo double



Conversiones explícitas

$$w := \text{tolong}(\text{todouble}(x) + y * \text{tofloat}(z))$$

w de tipo long

x de tipo integer

y de tipo float

z de tipo long

¡Ahora sí!



Conversiones explícitas

LR → AS → Assembler

Conversiones explícitas en el código fuente



Conversiones en el Árbol Sintáctico



Conversiones en Assembler

Conversiones explícitas

LR → T → Assembler

Conversiones explícitas en el código fuente



Conversiones en el los Tercetos



Conversiones en Assembler

Conversiones explícitas

LR → PI → Assembler

Conversiones explícitas en el código fuente



Conversiones en la Polaca Inversa



Conversiones en Assembler

Conversiones explícitas

LR → AS → T → Assembler

Conversiones explícitas en el código fuente



Conversiones en el Árbol Sintáctico



Conversiones en los Tercetos



Conversiones en Assembler

Conversiones explícitas

LR → AS → PI → Assembler

Conversiones explícitas en el código fuente



Conversiones en el Árbol Sintáctico



Conversiones en la Polaca Inversa



Conversiones en Assembler

¿Preguntas?

